

eBook-Template Repository Analysis: A Comparative Study of Multi-Format Document Generation Systems

WAFt Research Team, AI Assistant (Claude)

Abstract

This research document provides a comprehensive analysis of the eBook-Template repository, comparing its Asciidoctor-based multi-format eBook generation approach with WAFt's Python-native PDF template system. The analysis examines architectural differences, feature comparisons, integration opportunities, and provides actionable recommendations for both systems. Key findings reveal that while eBook-Template excels at multi-format output (PDF, ePub, Kindle), WAFt provides superior template diversity and Python integration. The research identifies complementary strengths and proposes integration strategies for enhanced document generation capabilities.

1. eBook-Template Repository Analysis

Date: 2026-01-12

Repository: <https://github.com/akosma/eBook-Template.git>

Status: Archived (May 22, 2025)

Purpose: Compare Asciidoctor-based eBook generation with WAFt's PDF systems

1.1 Executive Summary

The **eBook-Template** repository provides a comprehensive Asciidoctor-based workflow for generating multiple eBook formats (PDF, ePub, Kindle, HTML5, Unix man pages) from a single source. While

WAFt already has robust PDF generation capabilities, this template offers valuable patterns for multi-format output and structured document organization.

Key Insight: WAFt focuses on PDF generation with rich templating, while eBook-Template focuses on multi-format output from a single source. These are complementary approaches.

1.2 Architecture Comparison

eBook-Template Architecture

```
master.adoc (main file)
├─ chapters/ (individual chapter files)
├─ images/ (PNG images)
├─ code/ (code samples)
├─ data/ (XML, CSV data)
├─ _conf/ (configuration files)
├─ _resources/ (styles, templates)
└─ Makefile (build orchestration)
```

Technology Stack:

- **Asciidoctor** - Document processor (Ruby-based)
- **Asciidoctor-PDF** - PDF backend
- **Asciidoctor-EPUB3** - ePub backend
- **PlantUML** - UML diagram generation
- **Mathematical** - LaTeX math rendering
- **Rouge** - Syntax highlighting

Build Process:

```
make all           # Generates all formats
make pdf           # PDF only
make epub          # ePub only
make kindle        # Kindle (.mobi) only
make html          # HTML5 only
make clean         # Clean build artifacts
```

```
from waft import DocumentBuilder

# Template-based (production)
DocumentBuilder.field_guide(
    title="Guide",
    content=markdown_content
).generate()

# Foundation-based (experimental)
from waft.foundation_v2 import DocumentEngine
engine = DocumentEngine()
# ... block-based API
```

WAFT Architecture

```
src/waft/templates/
├─ field_guide.py
├─ academic_paper.py
├─ lab_notes.py
├─ personal_memo.py
└─ tm_report.py

src/waft/document_builder.py (unified API)
src/waft/foundation_v2.py (FPDF2 alternative)
```

Technology Stack:

- **WeasyPrint** - HTML to PDF conversion (primary)
- **FPDF2** - Pure Python PDF generation (alternative)
- **Jinja2** - Template engine
- **Markdown** - Content format (via markdown library)

Build Process:

1.3 Feature Comparison

Feature	eBook-Template	WAFT
PDF Generation	✔ Asciidoctor-PDF	✔ WeasyPrint (primary), FPDF2 (alt)
ePub Generation	✔ Native support	✘ Not supported
Kindle (.mobi)	✔ Native support	✘ Not supported
HTML5 Output	✔ Self-contained HTML	✔ Via WeasyPrint
Multi-file Support	✔ Asciidoctor includes	✔ Via DocumentBuilder

Feature	eBook-Template	WAFT
Syntax Highlighting	✅ Rouge (Swift, Kotlin, etc.)	✅ Via markdown extensions
UML Diagrams	✅ PlantUML integration	❌ Not supported
Math Rendering	✅ LaTeX via Mathematical	❌ Not supported
Template System	⚠️ AsciiDoctor attributes	✅ Jinja2 templates
Version Control	✅ Plain text (AsciiDoc)	✅ Markdown/HTML
Build System	✅ Makefile	✅ Python scripts
Template Library	❌ Single template	✅ Multiple templates (5+)
Template Registry	❌ Not applicable	✅ TemplateRegistry system

1.4 Strengths & Weaknesses

eBook-Template Strengths

1. **Multi-Format Output** ⭐
2. Single source generates PDF, ePub, Kindle, HTML
3. Consistent styling across formats
4. Industry-standard formats for eBook distribution
5. **Structured Organization**
6. Clear separation: chapters/, images/, code/, data/
7. Master file with includes
8. Version-control friendly (plain text)
9. **Rich Features**
10. PlantUML for diagrams
11. Mathematical for LaTeX equations
12. Syntax highlighting for many languages
13. Cross-references and TOC generation
14. **Mature Toolchain**
15. AsciiDoctor is well-established
16. Good documentation
17. Active community

eBook-Template Weaknesses

1. **Ruby Dependencies**
2. Requires Ruby, gems, system libraries

- 3. Complex installation (especially on macOS)
- 4. Docker option available but adds complexity

5. Limited Template Variety

- 6. Single template structure
- 7. Customization via AsciiDoctor attributes
- 8. Less flexible than Jinja2 templates

9. Archived Project

- 10. No longer maintained (archived May 2025)
- 11. May have compatibility issues with newer tools

12. Python Integration

- 13. Not Python-native
- 14. Would require subprocess calls
- 15. Less seamless than WAFT's Python-first approach

WAFT Strengths

- 1. **Python-Native** ★
- 2. Pure Python workflow
- 3. Easy integration with existing code
- 4. No external dependencies (except WeasyPrint system libs)

5. Template Library System

- 6. Multiple templates (field guide, academic, lab notes, etc.)
- 7. TemplateRegistry for discovery

- 8. Easy to add new templates

9. Flexible Templating

- 10. Jinja2 provides powerful templating

- 11. CSS-based styling
- 12. Easy customization

13. Active Development

- 14. Currently maintained
- 15. Evolving template system
- 16. PDF recreation capabilities

WAFT Weaknesses

1. Single Format Focus

- 2. PDF only (no ePub, Kindle)
- 3. Would need separate tooling for eBook formats

4. No Diagram Support

- 5. No PlantUML integration
- 6. Would need external diagram generation

7. No Math Rendering

- 8. No LaTeX equation support
- 9. Limited for academic/scientific documents

10. Markdown Limitations

- 11. Less structured than AsciiDoc
 - 12. No native include mechanism (though can be added)
-

1.5 Potential Integration Points

1. Multi-Format Output for WAFT

Opportunity: Add ePub/Kindle generation to WAFT

Approach:

- Use AsciiDoctor as a backend for multi-format output
- Keep WeasyPrint for PDF (current system)
- Add `DocumentBuilder.epub()` and `DocumentBuilder.kindle()` methods
- Convert WAFT templates to AsciiDoc format when needed

Implementation:

```
# Future API
DocumentBuilder.field_guide(
    title="Guide",
    content=markdown_content
).generate(format="pdf")      # Current
).generate(format="epub")    # New
).generate(format="kindle")  # New
).generate(format="all")     # New - all formats
```

Challenges:

- Requires Ruby/AsciiDoctor installation
- Template conversion (Jinja2 → AsciiDoc)
- Maintaining two template systems

2. Structured Document Organization

Opportunity: Adopt eBook-Template's folder structure

Approach:

- Organize WAFT documents with chapters/, images/, code/ folders
- Use master file pattern for long documents
- Support include mechanism in DocumentBuilder

Implementation:

```
# Future API
DocumentBuilder.from_structure(
    master_file="master.md",
    chapters_dir="chapters/",
    images_dir="images/",
    code_dir="code/"
).generate()
```

3. Diagram and Math Support

Opportunity: Add PlantUML and LaTeX math to WAFT

Approach:

- Integrate PlantUML for UML diagrams
- Add LaTeX math rendering (via Mathematical or similar)
- Support in templates via special blocks

Implementation:

```
# Future template support
content = """
# Chapter 1

[plantuml]
----
class User {
    +name: String
    +email: String
}
----

[math]
----
E = mc^2
----
"""
```

4. Build System Pattern

Opportunity: Add Makefile-based build for WAFT documents

Approach:

- Create Makefile templates for common WAFT workflows
- Support batch generation of multiple documents
- Clean build artifacts

Implementation:

```
# Future WAFT Makefile
all: pdf html

pdf:
    python -m waft.cli generate --template field_guides

html:
    python -m waft.cli generate --template field_guides

clean:
    rm -rf _build/
```

1.6 Recommendations

Short-Term (Keep Current Approach)

1. Continue with WeasyPrint + Jinja2

2. Production-ready and working
3. Python-native

4. Flexible templating

5. Enhance Template Library

6. Add more templates
7. Improve TemplateRegistry
8. Better template discovery

9. Document Best Practices

10. Multi-file document organization
11. Image/code asset management
12. Template customization guide

Medium-Term (Consider Integration)

1. **Add Multi-Format Support**
2. Evaluate Asciidoctor for ePub/Kindle
3. Or use alternative Python libraries (ebooklib, etc.)
4. Keep PDF generation with WeasyPrint
5. **Structured Document Support**
6. Add include mechanism to DocumentBuilder
7. Support master file pattern
8. Organize assets in folders
9. **Diagram Support**
10. Integrate PlantUML (Python library available)
11. Add diagram blocks to templates
12. Support in markdown processing

Long-Term (Full Integration)

1. **Hybrid Approach**
2. Use WAFT templates for PDF (current)
3. Use Asciidoctor for multi-format eBooks
4. Shared content source (Markdown → AsciiDoc conversion)
5. **Unified Build System**
6. Makefile for batch operations
7. Python CLI for programmatic use
8. Docker support for consistent builds
9. **Complete Feature Parity**
10. Math rendering

11. Diagram generation
12. Multi-format output
13. Structured organization

1.7 Conclusion

The **eBook-Template** repository offers valuable patterns for multi-format document generation and structured organization. However, WAFT's current Python-native approach with WeasyPrint and Jinja2 templates is well-suited for its primary use case (PDF generation).

Key Takeaways:

1. **Different Focus:** eBook-Template = multi-format eBooks, WAFT = rich PDF templates
2. **Complementary:** Could integrate Asciidoctor for ePub/Kindle while keeping WeasyPrint for PDF
3. **Patterns to Adopt:** Structured folder organization, master file pattern, build system
4. **Features to Consider:** PlantUML diagrams, LaTeX math, multi-format output

Recommendation: Continue with WAFT's current approach, but consider adding:

- Multi-format output (ePub/Kindle) via Asciidoctor or Python alternatives
 - Structured document organization patterns
 - Diagram and math support for academic/scientific documents
-

1.8 References

- **eBook-Template Repository:** <https://github.com/akosma/eBook-Template>
- **Asciidoctor Documentation:** <https://asciidoctor.org/>
- **WAFT Template System:** `src/waft/templates/`
- **WAFT DocumentBuilder:** `src/waft/document_builder.py`

• **WAFT Template Registry:** `src/waft/templates/registry.py`

Analysis Date: 2026-01-12
Analyst: AI Assistant (Claude)
Work Effort: WE-260112-q6gl (PDF Template Library System)

2. References

eBook-Template Repository: <https://github.com/akosma/eBook-Template>
Asciidoctor Documentation: <https://asciidoctor.org/>
WAFT Template System: `src/waft/templates/`
WAFT DocumentBuilder: `src/waft/document_builder.py`
WAFT Template Registry: `src/waft/templates/registry.py`